



El SDLC

Presentado por: Santiago Javela
Ospino

MAYO 2026

¿Qué es el SDLC?

ENTENDIENDO EL CICLO DE VIDA DEL SOFTWARE

El **Ciclo de Vida del Desarrollo de Software (SDLC)** es un marco estructurado que guía el proceso de planificación, diseño y mantenimiento del software, minimizando riesgos y optimizando recursos.

Las 7 Fases del SDLC

1. Planificación (Planning): Fase 1: Viabilidad y alcance. Es el punto de partida. Aquí se define el alcance del proyecto, se evalúa la viabilidad técnica y financiera, se calculan los costos, los tiempos y se asignan los recursos. El objetivo es responder: ¿Qué queremos construir y es realista hacerlo?
2. Análisis de Requisitos (Requirements Analysis): Fase 2: Qué debe hacer el sistema. Se recopila toda la información de los clientes y partes interesadas. Se definen las funciones, restricciones y metas del software. El resultado clave de esta fase es el documento SRS (Software Requirements Specification), que sirve como el contrato técnico del proyecto.
3. Diseño (Design): Fase 3: Cómo se va a construir. Los arquitectos de software traducen los requisitos en una estructura física. Se define la arquitectura del sistema, el modelo de la base de datos (tablas, relaciones), las interfaces de usuario (mockups/wireframes) y los flujos de datos.
4. Desarrollo / Codificación (Development): Fase 4: Escribir el código. La fase más larga. Los desarrolladores empiezan a picar código basándose en el diseño previo. Se eligen los lenguajes, frameworks y se implementan las buenas prácticas de control de versiones (como Git) y estándares de código limpios.
5. Pruebas / Control de Calidad (Testing): Fase 5: Buscar errores. El equipo de QA (Quality Assurance) evalúa el software para asegurar que cumple con los requisitos del SRS y que no tiene fallos (bugs). Se realizan pruebas unitarias, de integración, de rendimiento y de seguridad.
6. Despliegue / Implementación (Deployment): Fase 6: Lanzamiento a producción. Una vez que el software es estable, se libera en el entorno de producción para que los usuarios finales puedan utilizarlo. Puede hacerse de forma escalonada (fase beta) o un lanzamiento completo.
7. Mantenimiento y Evolución (Maintenance): Fase 7: Soporte continuo. El software ya está vivo. En esta fase se corrigen errores menores que no se detectaron en QA, se aplican parches de seguridad y se realizan actualizaciones para adaptar el sistema a nuevas necesidades del mercado o del cliente.

Por qué el Alcance es Fundamental

EVITANDO EL SCOPE CREEP EN PROYECTOS

Porque evita el "Scope Creep" (corrupción del alcance), que es cuando se añaden funciones de manera descontrolada durante el desarrollo. Definir bien el alcance permite estimar correctamente el presupuesto, los plazos de entrega y los recursos necesarios, evitando retrasos y sobrecostos.

Clasificación de Requisitos: Funcional y No Funcional

- A) EL SISTEMA DEBE PERMITIR INICIAR SESIÓN CON CORREO Y CONTRASEÑA.
- ➡ REQUISITO FUNCIONAL (RF): DEFINE UNA ACCIÓN ESPECÍFICA QUE EL SISTEMA DEBE HACER (AUTENTICACIÓN).
- B) EL SISTEMA DEBE RESPONDER EN MENOS DE 2 SEGUNDOS.
- ➡ REQUISITO NO FUNCIONAL (RNF): DEFINE UN ATRIBUTO DE CALIDAD O RENDIMIENTO (VELOCIDAD).
- C) EL SISTEMA DEBE PERMITIR REGISTRAR PAGOS MENSUALES.
- ➡ REQUISITO FUNCIONAL (RF): ES UNA CARACTERÍSTICA DEL NEGOCIO (GESTIÓN DE PAGOS).
- D) EL SISTEMA DEBE SOPORTAR 500 USUARIOS CONECTADOS AL MISMO TIEMPO.
- ➡ REQUISITO NO FUNCIONAL (RNF): DEFINE UNA RESTRICCIÓN TÉCNICA DE ESCALABILIDAD Y CAPACIDAD.

Diseño HLD vs LLD: Arquitectura General y Detalles

HLD: DISEÑO DE ALTO NIVEL

El HLD define la arquitectura general del sistema, incluyendo módulos y servicios. Es el plano de un arquitecto que muestra cómo se comunican las diferentes partes entre sí, asegurando un diseño cohesivo y eficiente.

LLD: DISEÑO DE BAJO NIVEL

El LLD se centra en los detalles específicos de cada componente. Incluye diagramas de clases y esquemas de base de datos, proporcionando una guía precisa para los ingenieros en la implementación del sistema.

Patron MVC: Modelo Vista Controlador

ESTRUCTURA DE CAPAS

El patrón MVC separa la aplicación en tres capas: Modelo, Vista y Controlador, lo que facilita la gestión del código y la colaboración entre equipos de desarrollo.

DESACOPLAMIENTO EFECTIVO

Este desacoplamiento permite que los cambios en la interfaz de usuario no afecten la lógica de negocio, lo que resulta en un desarrollo más ágil y fácil de mantener.

VENTAJAS CLAVE

La implementación de MVC conduce a un código más limpio y organizado, promoviendo mejores prácticas de desarrollo y aumentando la eficacia del equipo en la resolución de problemas.

Clean Code

PRINCIPIO KISS

El principio **KISS** (Keep It Simple Stupid) enfatiza que la solución más simple es la preferible, evitando complejidades innecesarias que pueden dificultar la comprensión y el mantenimiento del código.

NOMBRES SIGNIFICATIVOS

Utilizar **nombres significativos** para variables y funciones mejora la legibilidad del código. Nombres descriptivos como registrarPagoMensual son preferibles a términos vagos, facilitando la comprensión del propósito del código.

PRINCIPIO DRY

El principio **DRY** (Don't Repeat Yourself) promueve la eliminación de código duplicado. Cada pieza de conocimiento debe tener una única representación dentro del sistema, facilitando el mantenimiento y la evolución.

FUNCIONES PEQUEÑAS

Es recomendable escribir **funciones pequeñas** y con un propósito único. Esto no solo hace que el código sea más fácil de seguir, sino que también reduce la probabilidad de introducir errores.

Pruebas Unitarias: Validación de Código Aislado

IMPORTANCIA DE LAS PRUEBAS

Las pruebas unitarias son cruciales para validar funciones individuales, asegurando que cada parte del código funcione correctamente y minimizando errores antes de la implementación.

AUTOMATIZACIÓN DE PRUEBAS

Automatizar pruebas unitarias permite detectar regresiones rápidamente, garantizando que los cambios en el código no rompan funcionalidad existente, lo que ahorra tiempo y recursos en el desarrollo.

HERRAMIENTAS COMUNES

Herramientas como JUnit, pytest y Jest son populares en la comunidad de desarrollo, facilitando la creación y ejecución de pruebas unitarias en diferentes lenguajes de programación.

CICD y Contenedores en Desarrollo

INTEGRACIÓN CONTINUA

La **Integración Continua (CI)** permite que cada cambio de código se integre, compile y pruebe automáticamente, garantizando la calidad del software.

ENTREGA DESPLIEGUE CONTINUO

La **Entrega Despliegue Continuo (CD)** automatiza el despliegue del software validado, eliminando inconsistencias y asegurando que todo funcione correctamente en producción.

Mantenimiento Correctivo y Evolutivo en POWERTRACK

MANTENIMIENTO CORRECTIVO

El mantenimiento correctivo se enfoca en **corregir errores** y bugs detectados en producción que no se encontraron durante las pruebas. Por ejemplo, en POWERTRACK, un usuario puede no ver su historial de pagos debido a un error en la consulta SQL.

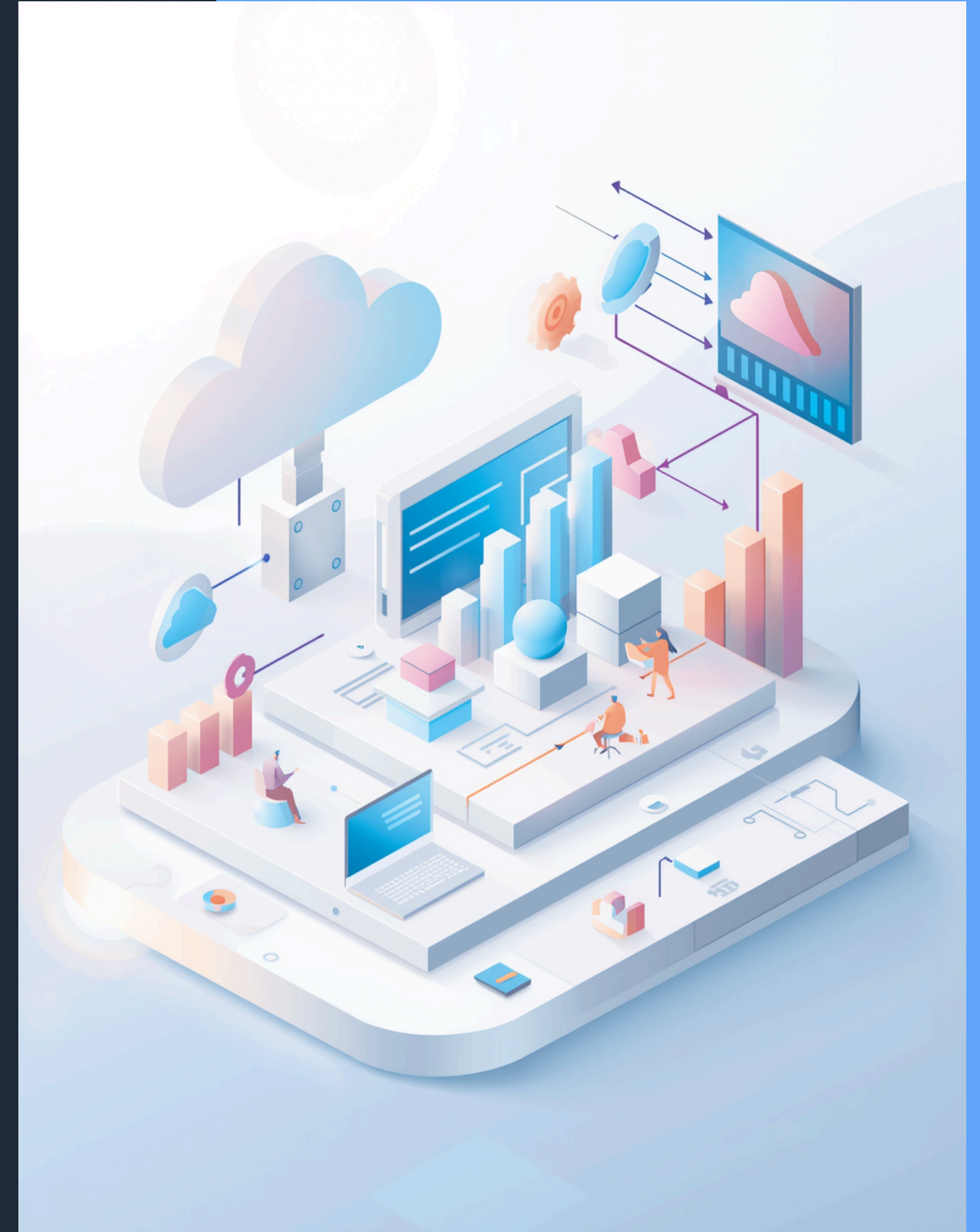
MANTENIMIENTO EVOLUTIVO

Por otro lado, el mantenimiento evolutivo tiene como objetivo **agregar nuevas funcionalidades** o mejorar las existentes para adaptarse a las necesidades del negocio. Un ejemplo es implementar el soporte para pagos con tarjeta de crédito que no existía en el lanzamiento inicial.

Lecciones Clave

- ¿Qué tipo de requisito falló?
- Falló un Requisito No Funcional (RNF) de rendimiento, escalabilidad o concurrencia.
- ¿En qué fase del SDLC debió detectarse ese problema?
- Debió detectarse en la Fase de Pruebas (Testing), específicamente al realizar Pruebas de Carga o de Estrés, las cuales sirven justamente para simular picos masivos de usuarios y verificar los límites del servidor antes de salir a producción.

– ANÁLISIS DE FALLA



Thanks

